AWS SAA-C03 Exam Notes: Targeted Topics

1. AWS CloudFormation

- **Concept:** Infrastructure as Code (IaC). You define your infrastructure in a JSON or YAML template, and CloudFormation provisions it.
- **Key Exam Scenarios:**
  - **Drift Detection:** Use this to detect if someone manually changed a resource (e.g., changed a Security Group rule) outside of the CloudFormation stack.
  - **StackSets:** Used to create, update, or delete stacks across *multiple accounts* and *AWS Regions* with a single operation.
  - **DeletionPolicy:** Controls what happens to a resource when the stack is deleted.
    - `Retain` : Keeps the resource (good for S3 buckets/databases you don't want to lose).
    - `Snapshot` : Creates a snapshot before deleting (for RDS/EBS).
  - **DependsOn:** Use this to control the creation order of resources (e.g., ensure an S3 bucket is created before the Lambda function that uses it).

2. CloudFormation - Service Role

- **The Problem:** A user (e.g., a Junior Dev) needs to create a stack that includes IAM roles or VPCs, but they don't have permission to create those specific resources directly.
- **The Solution:** Assign a **Service Role** to the CloudFormation Stack.
- **How it works:**
  1. An Admin creates an IAM Role (Service Role) with powerful permissions (e.g., `AdministratorAccess`).
  2. The Admin grants the Junior Dev permission to *PassRole* to CloudFormation.
  3. The Junior Dev launches the stack and selects this Service Role.
  4. CloudFormation uses the **Service Role's permissions** to create the resources, not the Junior Dev's permissions.
- **Exam Keyword:** "Least Privilege" for users while allowing them to deploy approved infrastructure.

3. Amazon SES (Simple Email Service)

- **Primary Use Case:** Sending and receiving **transactional emails** at scale.
- **Scenarios:**
  - Application needs to send password resets, order confirmations, or OTPs.
  - Marketing emails where you manage the list and content logic yourself.
  - **Hybrid:** specific need to send email from an on-premises app through AWS (uses SMTP interface).

- **Differentiation:** If the question mentions "marketing campaigns," "user engagement," or "analytics," look at Pinpoint instead. SES is the raw engine; Pinpoint is the marketing brain.

4. Amazon Pinpoint

- **Primary Use Case: Marketing communications**, user engagement, and segmentation.
- **Key Features:**
  - **Channels:** Sends Email, SMS, Push Notifications, and Voice messages.
  - **Segmentation:** "Send a coupon to all users who haven't opened the app in 30 days."
  - **Campaigns & Journeys:** Orchestrates multi-step user experiences.
- **Exam Hint:** Look for keywords like "marketing campaigns," "user behavior analysis," "targeted segments," or "push notifications."

5. SSM Session Manager (Systems Manager)

- **Concept:** Securely manage EC2 instances without opening inbound ports (No SSH/Port 22 required).
- **Architecture:**
  - Agent-based (SSM Agent must be installed/running).
  - Instance needs an IAM role with `AmazonSSMManagedInstanceCore`.
  - Traffic flows via AWS Systems Manager API (HTTPS), not SSH.
- **Exam Scenarios:**
  - "How to securely access an instance in a private subnet with no Bastion Host?" -> **Session Manager**.
  - "Need to audit who accessed an instance and what commands they ran?" -> **Session Manager** (logs session data to S3 or CloudWatch Logs).

6. SSM Other Services (High Probability)

- **SSM Parameter Store:**
  - Stores configuration data (database strings, license codes) and secrets (passwords).
  - **Free tier available** (Standard).
  - **Integration:** Works natively with CloudFormation, ECS, and Lambda.
  - *Vs. Secrets Manager:* Parameter Store is cheaper/free but lacks automatic rotation (unless you write a custom Lambda). Secrets Manager has built-in rotation.
- **SSM Patch Manager:**
  - Automates patching for managed instances (OS updates).
  - Uses **Patch Baselines** to define approved patches (e.g., "Critical updates only").
  - Can patch large fleets of EC2 instances and on-prem servers during a **Maintenance Window**.

- **SSM Run Command:**
  - Execute scripts/commands on a fleet of instances remotely and securely (no SSH).

7. AWS Cost Explorer

- **Primary Use Case:** Visualizing and analyzing **historical** cost and usage data.
- **Features:**
  - View costs for the last 12 months.
  - Forecast spend for the next 12 months.
  - **Rightsizing Recommendations:** Identifies underutilized EC2 instances (e.g., "You are using m5.large but only need m5.medium").
- **Exam Hint:** If the question asks to "analyze spending trends" or "forecast future costs," it's Cost Explorer.

8. AWS Cost Anomaly Detection

- **Primary Use Case: Automated monitoring** to detect unusual spending spikes using Machine Learning.
- **How it works:** It learns your baseline usage patterns. If costs spike unexpectedly (e.g., a developer accidentally left a large instance running), it sends an alert (SNS email).
- **Differentiation:**
  - **Cost Explorer:** You go to look at the data.
  - **Anomaly Detection:** It alerts you when something is wrong.
  - **AWS Budgets:** You set a static limit (e.g., "Alert me if spend > $100"). Anomaly Detection finds the "unknown unknowns."

9. AWS Outposts

- **Concept:** Hybrid Cloud. Run AWS infrastructure and services (EC2, EBS, RDS, EKS) **on-premises** in your own data center.
- **Physical Form:** A physical rack of servers delivered by AWS to your location.
- **Use Cases:**
  - **Low Latency:** Applications that need single-digit millisecond latency to on-prem machinery/systems.
  - **Data Residency:** Data must remain in a specific physical location for regulatory reasons (where no AWS Region exists).
- **Vs. Local Zones:** Outposts = Your Data Center. Local Zones = AWS infrastructure in a specific city (managed by AWS, not in your building).

10. AWS Batch

- **Primary Use Case:** Running hundreds/thousands of **batch computing jobs** efficiently.
- **Architecture:**

    - Dynamically provisions compute resources (e.g., EC2 Spot Instances) based on job volume.
    - Uses Docker containers.
- **Vs. Lambda:**
    - Lambda has a **15-minute timeout**.
    - If a job takes > 15 minutes, use **AWS Batch**.
    - If the workload involves complex dependencies or huge disk requirements, use **AWS Batch**

11. Amazon AppFlow

- **Concept:** Fully managed integration service to securely transfer data between **SaaS applications** and AWS.
- **Connectors:** Salesforce, ServiceNow, Slack, Zendesk, Marketo, Google Analytics.
- **Destinations:** S3, Redshift, Snowflake.
- **Exam Scenario:** "Company wants to analyze Salesforce data in Redshift. How to transfer the data with the least operational overhead?" -> **Amazon AppFlow**. (No coding required).

12. AWS Amplify

- **Target Audience:** Front-end web and mobile developers.
- **Function:** A set of tools to build, deploy, and host full-stack applications.
- **Key Features:**
    - **Amplify Hosting:** CI/CD for static web apps (connects to Git repo).
    - **Amplify Studio:** Visual interface to build backend (Auth, DataStore/Storage) without deep cloud expertise.
    - **Authentication:** Easy wrapper around Amazon Cognito.

13. Instance Scheduler on AWS

- **Important:** This is an **AWS Solution** (a pre-built CloudFormation template you deploy), not a standalone service in the console "Service" dropdown.
- **Function:** Automatically starts and stops EC2 and RDS instances on a schedule (e.g., Stop at 6 PM Friday, Start at 8 AM Monday).
- **Architecture (Exam setup):**
    - Uses **Amazon EventBridge** to trigger a **Lambda function**.
    - Configuration (schedules) is stored in **DynamoDB**.
    - Tags are used to identify which instances to schedule.
- **Benefit:** drastic cost reduction for non-production environments (dev/test).